Algorithmics and Data Structures

Session 4
Graph and Trees

Ali Ismail-Fawaz

ENSISA, Université Haute-Alsace

Overview

What We'll Cover Today:

- **Introduction to Trees**
  - Definition and Properties
  - Binary Trees and Binary Search Trees (BST)
- **Introduction to Graphs**
  - Graph Terminology and Types (Directed, Undirected, Weighted)
  - Graph Representations: Adjacency List vs. Matrix
- **Graph Traversal Algorithms**
  - Breadth-First Search (BFS)
  - Depth-First Search (DFS)

Contents

Trees

Binary Trees

Graphs

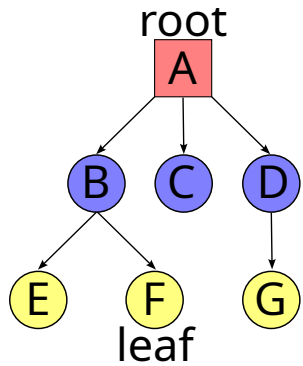
Introduction to Trees

What is a Tree?

- A hierarchical data structure made up of nodes
- One special node is called the **root**
- Each node may have child nodes, forming parent-child relationships
- Nodes with no children are called **leaves**
- Each child has exactly one parent (except the root, which has none)

Common Uses:

- Organizing hierarchical data (e.g., file systems)
- Efficient searching and sorting (e.g., Binary Search Trees)
- Parsing expressions and syntactic structures

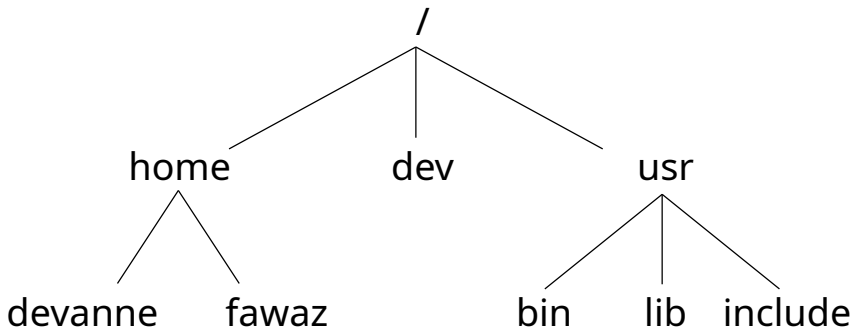


Properties of Trees

- **Number of Nodes and Edges**:
  - A tree with n nodes has exactly $n - 1$ links between nodes
- **Level of a Node**:
  - The distance (in number of edges) from the root to the node
  - The root is at level 0
- **Height of a Tree**:
  - The maximum level of any node in the tree
  - Also known as the depth of the deepest node
- **Sub-tree**:
  - Any node and its descendants form a sub-tree

Example: File System

A tree can represent the hierarchy of a file system

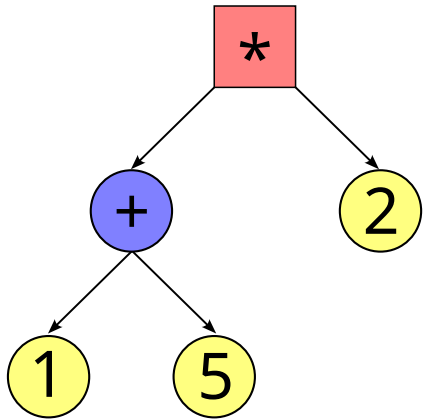


Applications of Trees

- **Mathematical Expressions:**
  - Syntax trees are used to represent and evaluate arithmetic expressions
- **Networking and Routing:**
  - Minimum spanning trees (ex: Prim's algorithm) help optimize network design
- **Programming Languages:**
  - Abstract Syntax Trees (ASTs) are used by compilers to analyze code structure
- **Games and Artificial Intelligence:**
  - Decision trees model possible choices and outcomes in strategic scenarios

Example: Mathemtical Expression

The mathematical expression $(1 + 5) * 2$ can be represented using the following syntax tree:



Contents

Trees

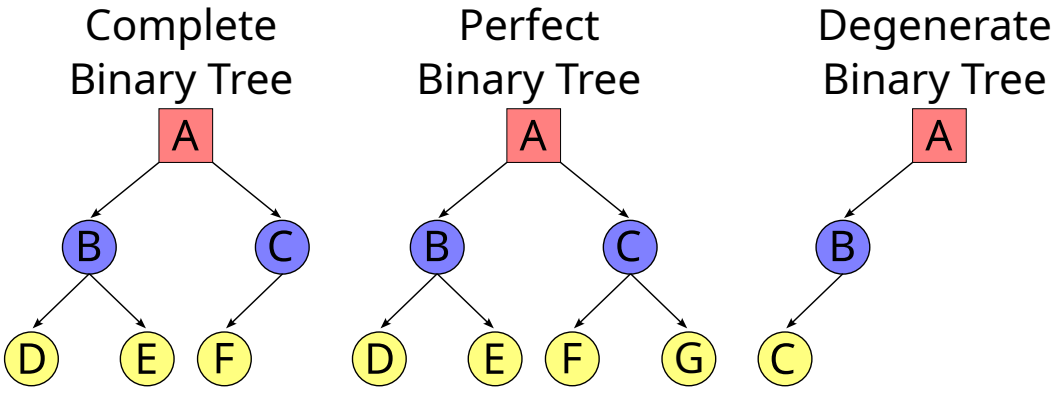
Binary Trees

Graphs

What is a Binary Tree?

- A binary tree is a tree in which each node has at most two children
- These children are usually referred to as the **left child** and the **right child**
- Binary trees are widely used in searching, sorting, expression evaluation, and more
- **Complete Binary Tree:**
  - All levels are fully filled **except possibly the last**
  - Nodes in the last level are placed as far **left as possible**
- **Perfect Binary Tree:**
  - All internal nodes have exactly **two children**
  - All leaf nodes are at the **same level**
- **Degenerate (Skewed) Binary Tree:**
  - Each node has **only one child**
  - The structure is similar to a **linked list**
  - Can be either left-skewed or right-skewed

Example of Binary Trees



Binary Search Tree (BST)

Definition:

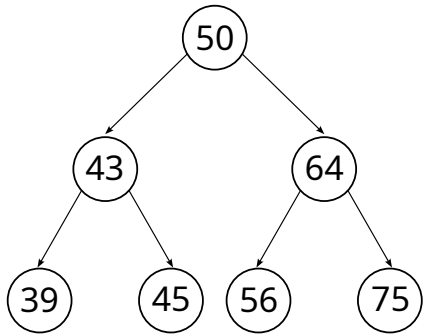
- A Binary Search Tree (BST) is a binary tree where:
  - The left sub-tree of a node contains only values **less than** the node's value
  - The right sub-tree contains only values **greater than** the node's value
  - This property holds **recursively** for all nodes

Advantages:

- Efficient search, insertion, and deletion in average case: $\mathcal{O}(\log n)$ for balanced trees

Disadvantages:

- In the worst case (unbalanced), operations degrade to $\mathcal{O}(n)$



Unbalanced BSTs and AVL Trees

- When data is inserted in sorted or nearly sorted order, a BST can become **highly unbalanced**
- This leads to a structure similar to a linked list
- Search, insertion, and deletion degrade to $\mathcal{O}(n)$ in the worst case

Solution: AVL Tree (Adelson-Velsky and Landis Tree)

- A self-balancing BST
- Maintains a balance factor for each node:
  - Balance Factor = Height(left sub-tree) − Height(right sub-tree)
  - Must be -1, 0, or $+1$
- Performs **rotations** (single or double) after insertion/deletion to maintain balance
- Guarantees $\mathcal{O}(\log n)$ time for all basic operations

Check out this resource for more details on AVL Trees: https://avlvisualizer.com/

Decision Trees

Definition:

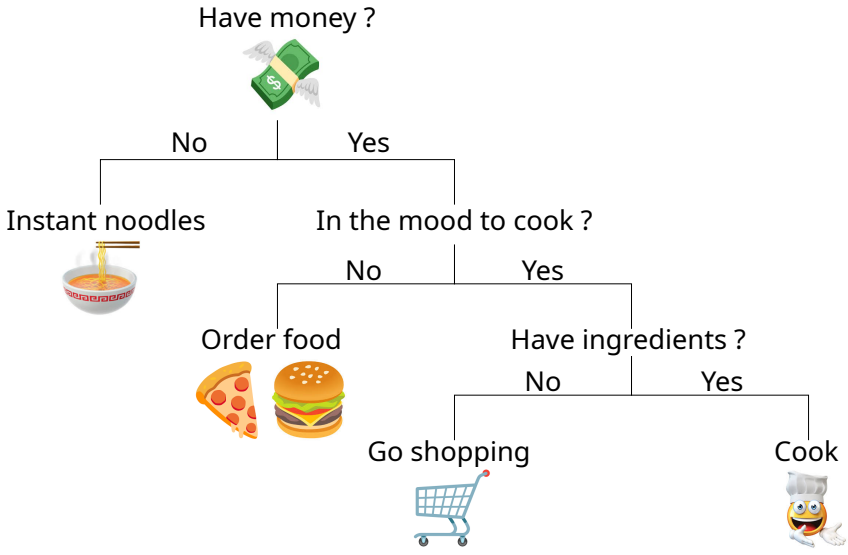
- A Decision Tree is a tree-based model used to make decisions or classify data
- Internal nodes represent **questions or tests**
- Edges represent **possible outcomes** of a test
- Leaf nodes represent **decisions**, **classes**, or **results**

Applications:

- Machine Learning (classification and regression)
- Game AI (ex: move selection)
- Expert systems and decision support tools

Example Decision Tree

College student deciding what to eat for dinner



Contents

Trees

Binary Trees

Graphs

Introduction to Graphs

Definition:

- A graph is a collection of **nodes (vertices)** connected by **edges**
- Used to model pairwise relationships between objects

Common Types of Graphs:

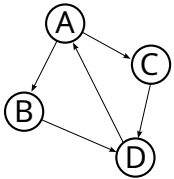
- **Directed vs. Undirected:** Edges have direction or not
- **Weighted vs. Unweighted:** Edges may carry numerical values
- **Cyclic vs. Acyclic:** Whether paths can loop back
- **Connected vs. Disconnected:** Whether all nodes are reachable

Note:

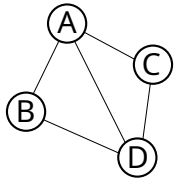
- A **tree is a special kind of graph**:
  - It is **directed connected** and has **no cycles**
  - Every pair of nodes is connected by exactly one unique path

Common Types of Graphs

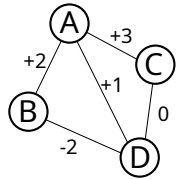
Directed



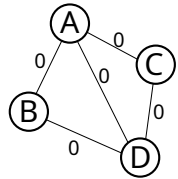
Undirected



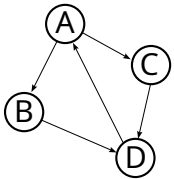
Weighted



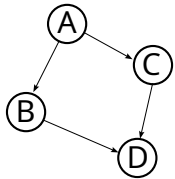
Unweighted



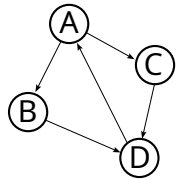
Cyclic



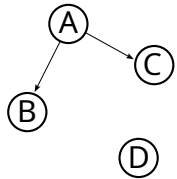
Acyclic



Connected



Disconnected



Graph Representations

1. **Adjacency Matrix**:
   - A 2D array of size $V \times V$ where V is the number of vertices
   - Entry (i, j) is 1 (or weight) if there is an edge from vertex i to vertex j, otherwise 0
   - **Pros**: Fast edge lookup $\mathcal{O}(1)$
   - **Cons**: Uses $\mathcal{O}(V^2)$ space, inefficient for sparse graphs

Example Graph:
Vertices: A, B, C Edges: A-B, A-C

Adjacency Matrix:

$$\begin{bmatrix} & A & B & C \\ A & 0 & 1 & 1 \\ B & 1 & 0 & 0 \\ C & 1 & 0 & 0 \end{bmatrix}$$

Graph Representations

1. **Adjacency Matrix**:
   - A 2D array of size $V \times V$ where V is the number of vertices
   - Entry (i, j) is 1 (or weight) if there is an edge from vertex i to vertex j, otherwise 0
   - **Pros**: Fast edge lookup $\mathcal{O}(1)$
   - **Cons**: Uses $\mathcal{O}(V^2)$ space, inefficient for sparse graphs

2. **Adjacency List**:
   - Each vertex stores a list of its adjacent vertices
   - More space-efficient for sparse graphs
   - **Pros**: Uses only $O(V + E)$ space
   - **Cons**: Slower edge lookup in the worst case

Example Graph:
Vertices: A, B, C Edges: A-B, A-C

Adjacency Matrix:

$$\begin{bmatrix} & A & B & C \\ A & 0 & 1 & 1 \\ B & 1 & 0 & 0 \\ C & 1 & 0 & 0 \end{bmatrix}$$

Adjacency List:
- A $\rightarrow$ [B, C]
- B $\rightarrow$ [A]
- C $\rightarrow$ [A]

Graph Traversal: BFS vs DFS

Graph traversal:

- The process of visiting all vertices in a graph
- Two main strategies: Breadth-First Search (BFS) and Depth-First Search (DFS)

Breadth-First Search (BFS):

- Explores the graph level by level
- Uses a **queue** (FIFO)
- Finds the shortest path in unweighted graphs

Depth-First Search (DFS):

- Explores as far as possible along a branch before backtracking
- Uses a **stack** (or recursion)
- Useful for detecting cycles, topological sorting, maze solving

Graph Traversal: BFS Pseudocode

Breadth-First Search (BFS)

Input: graph G, starting vertex v
Output: list of visited vertices
Instructions:

1. Mark v as visited and enqueue it in Q
2. While Q is not empty:
    a. $u \leftarrow$ Dequeue from Q
    b. For each neighbor n of u:
        i If n is not visited:
            Mark n as visited
            Enqueue n in Q

Graph Traversal: DFS Pseudocode

Depth-First Search (DFS)

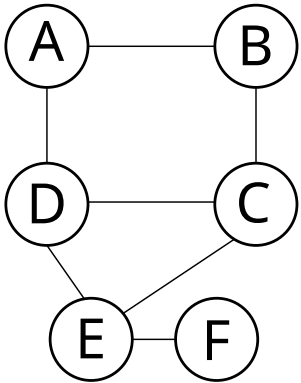
Input: graph G, starting vertex v
Output: list of visited vertices
Instructions:

1. Mark v as visited
2. For each neighbor n of v:
   a. If n is not visited:
   b. Call DFS(G, n)

BFS vs DFS: Comparison Table

Aspect	BFS	DFS
Time Complexity	$\mathcal{O}(V + E)$	$\mathcal{O}(V + E)$
Space Complexity	$\mathcal{O}(V)$ (due to queue)	$\mathcal{O}(V)$ (due to recursion or stack)
Advantage	Finds shortest path in unweighted graphs	Uses less memory on sparse graphs
Disadvantage	Uses more memory on wide graphs	May go deep unnecessarily
When to Use	Shortest path, level-order traversal	Topological sort, cycle detection
When Not to Use	Deep or infinite graphs (ex: web crawlers)	Very deep graphs with risk of stack overflow

Example DFS

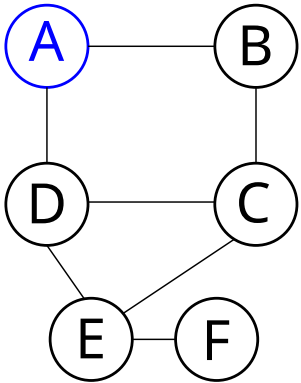


Visited / Not Visited

A	B	C	D	E	F

Output DFS:

Example DFS

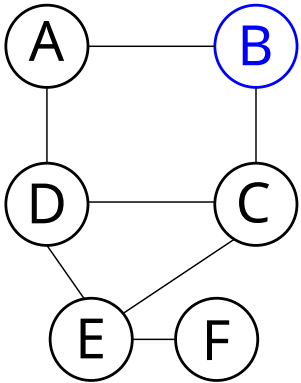


Visited / Not Visited

A	B	C	D	E	F

Output DFS: A

Example DFS

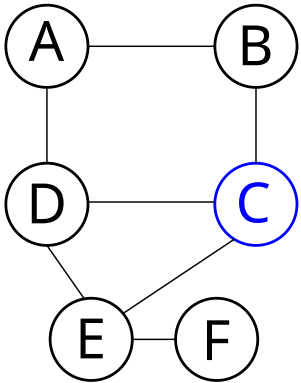


Visited / Not Visited

A	B	C	D	E	F

Output DFS: A, B

Example DFS

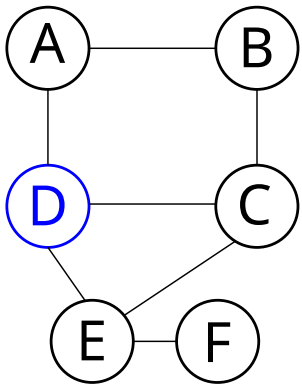


Visited / Not Visited

A	B	C	D	E	F

Output DFS:A, B, C

Example DFS

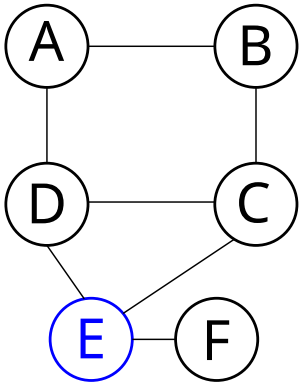


Visited / Not Visited



Output DFS:A, B, C, D

Example DFS

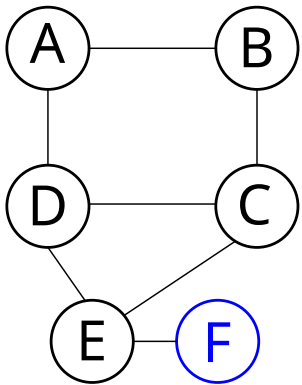


Visited / Not Visited



Output DFS:A, B, C,D, E

Example DFS

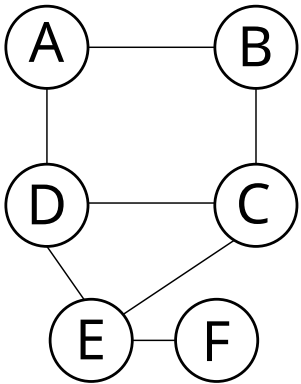


Visited / Not Visited

A	B	C	D	E	F

Output DFS:A, B, C, D, E, F

Example BFS



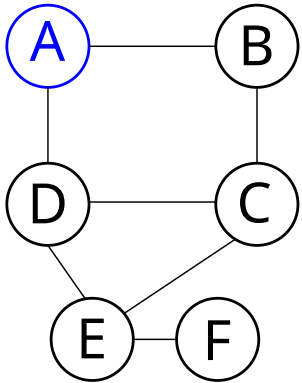
Visited / Not Visited

A	B	C	D	E	F

Queue: A

Output BFS:

Example BFS



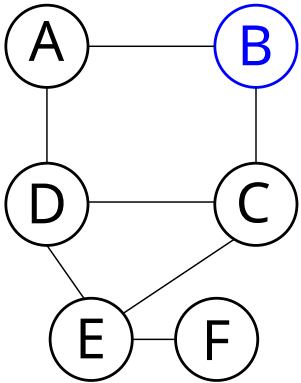
Visited / Not Visited

A	B	C	D	E	F

Queue: A

Output BFS: A

Example BFS



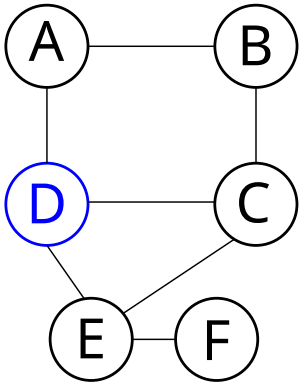
Visited / Not Visited

A	B	C	D	E	F

Queue: B

Output BFS: A, B

Example BFS



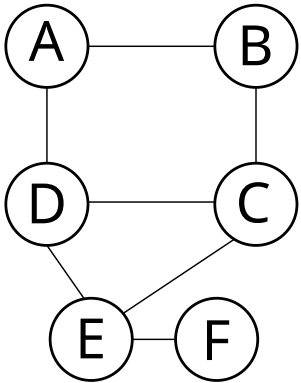
Visited / Not Visited

A	B	C	D	E	F

Queue: D, B

Output BFS: A, B, D

Example BFS



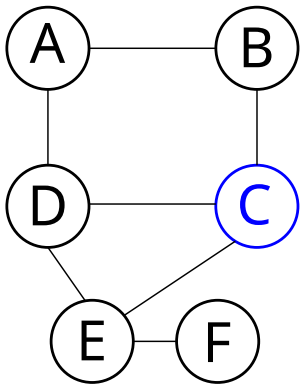
Visited / Not Visited

A	B	C	D	E	F

Queue: D

Output BFS: A, B, D

Example BFS



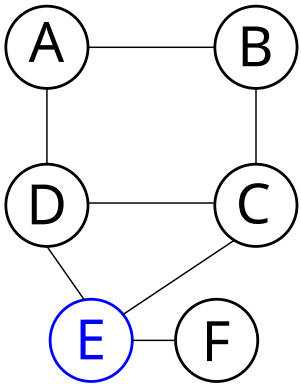
Visited / Not Visited

A	B	C	D	E	F

Queue: C, D

Output BFS: A, B, D, C

Example BFS



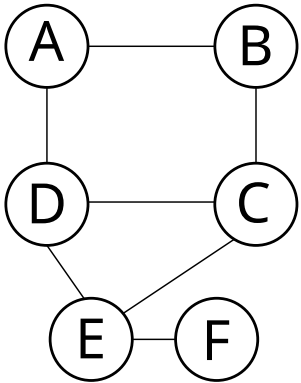
Visited / Not Visited

A	B	C	D	E	F

Queue: E, C

Output BFS: A, B, D, C, E

Example BFS



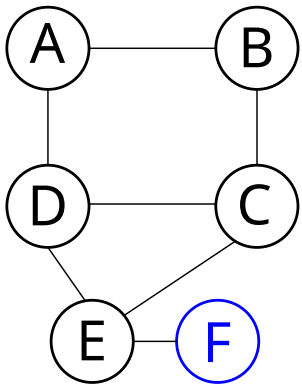
Visited / Not Visited

A	B	C	D	E	F

Queue: E

Output BFS: A, B, D, C, E

Example BFS



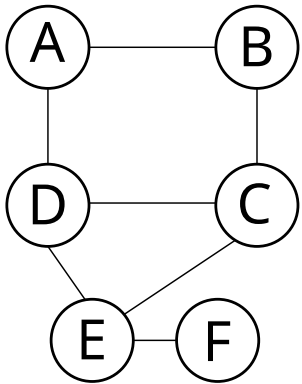
Visited / Not Visited

A	B	C	D	E	F

Queue: F

Output BFS: A, B, D, C, E, F

Example BFS



Visited / Not Visited

A	B	C	D	E	F

Queue:

Output BFS: A, B, D, C, E, F